

Interfaces

1 Motivation

Mit dem folgenden Code definieren wir die Klassen `Teacher` und `Student`. Beide Klassen haben eine `greet`-Methode.

```
record Teacher(String firstName, String lastName) {  
  
    public void greet() {  
        println("Hello, I am " + firstName + " " + lastName);  
    }  
}  
  
record Student(String firstName, String lastName, int age) {  
  
    public void greet() {  
        println("Yo, I am " + firstName);  
    }  
}
```

```
var herrMüller = new Teacher("Dieter", "Müller");  
herrMüller.greet();
```

Hello, I am Dieter Müller

```
var alex = new Student("Alex", "Hoffmann", 18);  
alex.greet();
```

Yo, I am Alex

Wenn sich ein Schüler zweimal vorstellen soll, kann man dafür folgende Methode schreiben:

```
class Utils {
    static void letStudentGreetTwoTimes(Student student){
        student.greet();
        student.greet();
    }
}
```

```
Utils.letStudentGreetTwoTimes(alex);
```

Yo, I am Alex

Yo, I am Alex

Obwohl die Methode für Lehrer genauso aufgebaut wäre, kann sie nur mit Schülern aufgerufen werden.

```
Utils.letStudentGreetTwoTimes(herrMüller); // Typfehler
```

Für Lehrer braucht man deshalb eine zweite Methode.

```
class Utils {
    static void letTeacherGreetTwoTimes(Teacher teacher){
        teacher.greet();
        teacher.greet();
    }
}
```

```
Utils.letTeacherGreetTwoTimes(herrMüller);
```

2 Interfaces

Um diese Codewiederholung zu vermeiden, kann man ein *Interface* definieren.

```
interface Greetable {
    void greet();
}
```

Jede Klasse, die das Interface `Greetable` implementiert, muss die Methode `greet` implementieren. Die Methode hat keine Parameter und liefert keinen Rückgabewert.

Wir können jetzt deklarieren, dass die Klasse `Teacher` das Interface implementiert. Das geschieht mit dem Schlüsselwort `implements`.

```
record Teacher(String firstName, String lastName) implements Greetable {
```

Vor jeder Implementierung einer Interface-Methode sollte `@Override` stehen.

```
@Override
    public void greet() {
        println("Hello, I am " + firstName + " " + lastName);
    }
}
```

Genauso implementiert auch Student das Interface.

```
record Student(String firstName, String lastName, int age) implements Greetable {
    @Override
    public void greet() {
        println("Yo, I am " + firstName);
    }
}
```

Student und Teacher können jetzt als Greetable-Referenzen gespeichert werden.

```
Greetable greetingAlex = alex;
Greetable greetingMüller = herrMüller;
```

Über diese Referenzen können weiterhin die Methoden des Interfaces verwendet werden.

```
greetingAlex.greet();
```

```
Yo, I am Alex
```

Andere Methoden der konkreten Klasse sind dann nicht mehr direkt nutzbar.

```
greetingAlex.firstName(); // Typfehler
```

Wenn wir Greetable als Parametertyp nutzen, können wir die beiden Methoden oben zu einer Methode zusammenfassen.

```
class Utils {
    static void letGreetTwoTimes(Greetable greetable){
        greetable.greet();
        greetable.greet();
    }
}
```

```
Utils.letGreetTwoTimes(greetingAlex);  
Utils.letGreetTwoTimes(greetingMüller);
```

```
Yo, I am Alex  
Yo, I am Alex  
Hello, I am Dieter Müller  
Hello, I am Dieter Müller
```

3 Interfaces und Listen

Die Objekte alex und herrMüller gehören zu unterschiedlichen Klassen, erfüllen aber beide das Interface Greetable. Die Objekte alex und herrMüller gehören zu unterschiedlichen Klassen, implementieren aber beide das Interface Greetable. Deshalb können sie in einer List<Greetable> gespeichert werden.

```
List<Greetable> greeters = List.of(alex, herrMüller);
```

So kann man Methoden definieren, die eine Liste von Objekten erhalten, die ein Interface implementieren.

```
class Utils {  
    static void showGreetings(List<Greetable> greetables) {  
        for (Greetable greetable : greetables) {  
            greetable.greet();  
        }  
    }  
}
```

```
Utils.showGreetings(greeters);
```

```
Yo, I am Alex  
Hello, I am Dieter Müller
```

4 Typen, die ein Interface implementieren, präzisieren

Eine Methode mit dem Parametertyp List<Greetable> kann zwar gemischte Listen verarbeiten, aber keine List<Student> oder List<Teacher> direkt annehmen.

Wenn gemischte Listen nicht erlaubt sein sollen und die Methode trotzdem sowohl mit einer List<Student> als auch mit einer List<Teacher> funktionieren soll, verwenden wir einen Typparameter T und begrenzen ihn mit extends Greetable.

Das bedeutet: T darf nur ein Typ sein, der Greetable implementiert.

```
static <T extends Greetable> void showGreetingsNotMixed(List<T> greetables) {
    for (Greetable greetable : greetables) {
        greetable.greet();
    }
}
```

```
var students = List.of(new Student("Alex", "Hoffmann", 18), new Student("Nino", "Clermont"));
var teachers = List.of(new Teacher("Dieter", "Müller"), new Teacher("Klaus", "Müller"));

Utils.showGreetingsNotMixed(students);
Utils.showGreetingsNotMixed(teachers);
```

```
Yo, I am Alex
Yo, I am Nino
Hello, I am Dieter Müller
Hello, I am Klaus Müller
```

Der gleiche Typparameter kann auch für den Rückgabewert genutzt werden. So bleibt der konkrete Typ erhalten: Bei einer `List<Student>` liefert die Methode ein `Student`-Objekt zurück, bei einer `List<Teacher>` ein `Teacher`-Objekt.

```
static <T extends Greetable> T greetAndReturnFirst(List<T> greetables) {
    T first = greetables.getFirst();
    first.greet();
    return first;
}
```

```
Student firstStudent = Utils.greetAndReturnFirst(students);
Teacher firstTeacher = Utils.greetAndReturnFirst(teachers);
```

```
Yo, I am Alex
Hello, I am Dieter Müller
```

Die letzten beiden Zuweisungen wären nicht möglich, wenn der Rückgabebetyp nur `Greetable` wäre. Dann ginge die Information über den konkreten Typ verloren.

5 Interface-Erweiterung

Wir können auch Interfaces definieren, die ein bestehendes Interface erweitern.

```
interface CanGreetAndSayGoodbye extends Greetable {
    void sayGoodbye();
}
```

Damit eine Klasse das Interface CanGreetAndSayGoodbye implementiert, muss sie auch Greetable implementieren und zusätzlich die Methode void sayGoodbye() implementieren.

```
record Student(String firstName, String lastName, int age) implements CanGreetAndSayGoodbye {
    @Override
    public void greet() {
        println("Yo, I am " + firstName);
    }

    @Override
    public void sayGoodbye() {
        println("San Frantschüssko");
    }
}
```

6 UML

Interfaces können auch in UML dargestellt werden. Sie werden wie Klassen als Rechteck dargestellt. Über dem Namen des Interfaces steht <<interface>>. Klassen, die das Interface implementieren, sind mit diesem durch einen Pfeil mit weißer Spitze verbunden. Die Klassen zeigen dadurch auf das Interface.

```
interface Greetable {
    void greet();
}

record Teacher(String firstName, String lastName) implements Greetable {
    @Override
    public void greet() {
        println("Hello, I am " + firstName + " " + lastName);
    }
}

record Student(String firstName, String lastName, int age) implements Greetable {
    @Override
    public void greet() {
        println("Yo, I am " + firstName);
    }
}
```

